Simplifying Rocq and Lean proofs by sharing semantic interfaces

Proof simplification research

21 October 2026

Abstract

The most promising next simplification is to reuse each language's stronger abstraction, rather than shrink another graph. Rocq can factor its duplicate certificate simulations through a semantic bisimulation relation already used in Lean. Lean can investigate the direct fresh-label seed construction already used in Rocq, avoiding an intermediate quotient rather than shortening its proof. New runtime work measures how much remains after the now-certified decoder prefix. These proposals must preserve exact source and endpoint attachments. No new proof-assistant compilation, machine reduction or measured build speedup is claimed here.

1 What the recent commits actually establish

The reviewed main pin is

`6f0d42d368506bc29c70ace1c992ed88b2651346`

It contains fifty commits since the preceding review. Current Lean uses the normalized 1,426-entry source certificate, explicit 512-fuel decoding and a separated stable program/tree module. Shared seed semantics and semantic roundtrips have already been adopted. The previous model-induction patch is therefore no longer an open opportunity.

Current Rocq has a different boundary. Its current source imports the older offset certificate and checks both directions. Its enumeration module is likewise not the newer decode-based version. A historical normalized Rocq 9.1 endpoint receipt records 44 fresh modules and recursive checking, but its source and enumeration hashes differ from the current files. That receipt cannot certify the current bytes; this mismatch does not show that the older offset proof is incorrect. It shows that the shortening was not retained at the current facade. The first integration task is to restore the exact checked closure or rebuild the reconciled version, not to attach an old PASS label to a new tree. Current dominated-implication code also retains the older 735-instruction reflective replay.

Lean's phase-9 record covers all nine freshly checked runtime batches, with 4,751 leaves and 6,176 outcomes. Its endpoint closure has 192 modules; the selected union has 197 including five decoder-prefix modules, partitioned into 20 fresh providers, 13 isolated reuses and 164 earlier reuses. The accepted record repairs an earlier transitive-dependency reuse error. Historical batch times are not a controlled paired benchmark. A later representation-independent 58-step implication core has focused checks; phase-9 endpoint evidence must not automatically be extended to changed dependency bytes.

The literal decoder-to-primitive zipper and a generic Home return theorem are already checked. Reattaching the decoder to its source or proving Home return again would repeat completed work. Primitive and continuation coverage remain separate.

1.1 Completed alternatives versus maintained adoption

The delivered CM215 archive now contains a complete historical Lean endpoint, not merely a proposed final wrapper. Its baseline uses 180 exact pinned sources and four explicit proof-only batching overlays; seven CM modules and an independent typed endpoint client are compiled. The client connects the exact machine to finite ZFC refutation without a consistency premise. Independent source/log inspection matches 2,980 recorded hashes, including repeated ancestry bindings, not 2,980 distinct compiled modules. Failed memory attempts and the four overlays are disclosed. These are inspected compiler records, not local compilation or proof of current-main integration.

Archived Rocq evidence also already checks the balanced-normalized 1,426-entry composition and the 367-node affine source core. Their final production endpoint attachments have separate status. They must not be presented as generally uncompiled new proposals. The new opportunity below is to simplify their shared semantic boundary.

2 Factor Rocq certificate soundness through a semantic relation

Ordinary and normalized Rocq certificate modules repeat 50- and 53-line simulation blocks. Their main 32-line induction is byte-identical after renaming checker-specific lemmas. The difference belongs at the branch interface: literal equality in one checker and implicit-zero endpoint equality in the other. Lean already separates these concerns with semantic bisimulation.

A Rocq interface can state that related program pointers agree on halting and, for each branch, execute compatible finite segments into related successors. Prove the all-register halting transport once from this relation. Ordinary, normalized and balanced checkers then establish the relation, instead of each copying the full execution induction. This is a port of existing proof organization, not a new notion of bisimulation.

Certificate lookup also needs less than layout validity for soundness. If every successful lookup returns a member of a collection whose entries are checked, and the initial entry is retrieved, the proof can use membership directly. Global size, ordering or balanced-tree validity may remain useful for construction and performance, but should not be required by the semantic transport theorem when lookup membership suffices. Missing lookups must fail, and the checker must still verify every returned entry and its successors.

Normalized representatives require one important adapter lemma: equivalent zero-padded paths have the same site lookup, including the consumed leaf path. Increment and break act on that consumed path; arbitrary raw pointers must not be substituted merely because they are zero-padding equivalent. The decrement test provides a real progress step even when the following increment segment is empty.

The normalized path utilities can be extracted into a small independent module. The affine graph bridge uses these utilities, not the entire certificate simulation. An owned textual check preserves the utility proofs and verifies that the bridge change only alters its import and qualification. This reduces a proof dependency without claiming a faster build. The proposed refactor and extraction remain uncompiled; compile their concrete clients and compare public theorem statements and assumptions before adoption.

3 Port the direct seed bridge from Rocq to Lean

The current Lean lower chain constructs an equality-free seed, passes to an indiscernibility quotient, and obtains a full seed there. Existing Rocq `Complete222` instead uses a private fresh label and obtains

the needed seed on the original domain. Lean's existing completeness interface supplies satisfaction of named formulas with *literal equality*, even though its syntactic premise uses the equality-eliminating translation. This makes the Rocq route applicable without a new completeness theorem or a circular import.

For any named formula ψ, including equality, define

$$p_\psi = \neg\neg\psi, \qquad c_\psi = \mathrm{enc}(\mathrm{th}(p_\psi)), \qquad B_\psi = \mathrm{bundleNF}(0, 1, c_\psi, p_\psi).$$

Two elementary support lemmas suffice for freshness: a free name is smaller than its formula's encoding, and translation preserves free support. If c_ψ were free in p_ψ, it would be free in its translation and hence strictly smaller than itself. The double-negation padding gives $c_\psi > 1$, keeping it distinct from the frozen roles zero and one. This is a statement about *free* names, not all syntactic occurrences.

Translation commutes with the named bundle because its added atoms are membership atoms. Consequently the encoding of $\mathrm{th}(B_\psi)$ is the enabled numerical bundle instance at c_ψ. Apply the existing named-theory satisfaction premise to B_ψ. A named bundle semantic lemma, with the fresh-label and distinct-role hypotheses, yields the seed property for p_ψ. Existing pointwise seed congruence removes double negation, giving the full seed for ψ on the original domain.

The named bundle lemma needs no irreflexivity assumption: named falsity is actual False. The existing numerical bundle theorem still needs irreflexivity because its falsity code is diagonal membership. Keep that numerical theorem, malformed-code cases and coincident-role branches unchanged. The final FullSeed structure still includes irreflexivity.

Only the first quotient can leave the endpoint path. The lower conversion consumes only the FullSeed interface, so its theorem need not change. Replace the full-seed constructor and the endpoint's quotient witness by the original witness $e(0)$. This preserves nonemptiness rather than silently assuming an inhabited domain. Ordinary completeness, the later tagged domain, its bisimulation quotient, the well-founded part and the final ZF model construction all remain. No reduction in final axioms follows merely from removing this preliminary quotient.

The current indiscernibility module has 151 lines, but deleting its implementation is not a net line-saving calculation: new support and named-semantic proofs must be added. Prefer sharing the existing semantic normalization over copying it. Compile the direct bridge, preserve the unchanged endpoint statement and assumption report, then prune imports and measure the actual result. This is a mathematically checked proposal, not a compiled Lean patch. Rocq already uses this route; no analogous Rocq quotient deletion is claimed.

4 Compose the checked decoder with residual site contracts

The checked decoder prefix is a primitive-entry theorem, not complete primitive or continuation coverage. Its clock may be zero, so it cannot by itself establish the positive progress required by the source simulation. The appropriate next interface is a residual contract from that exact zipper boundary to the existing postcondition, composed with the prefix theorem and the already-shared Home return.

An independent binding parses the current 202-node source and 234-role arrays and checks all 4,751 source paths, including history, child, opcode, operand and primitive action. Every numerical port matches the formal routed control state under the exact label map. This binds the experiment to the existing prefix interface without recompiling its theorem.

The owned symbolic experiment checks all 6,176 guarded outcomes on the exact formal K215 table. With the Home macro used on both sides, full-site execution takes 926,351 symbolic iterations; starting

at the primitive entry takes 705,880. The difference is 220,471 iterations, about 23.8% of this defined counter. Residual lengths range from one to 207. This is not a wall-clock benchmark, a new kernel proof, or a claim that primitive guards disappear. Existing BoxLemmas are assumed in this experiment.

An additional interface change is essential: current `KSim.Covered` literally requires the old Boolean `checkEdge` to return true. Semantic prefix reach plus residual success does not prove that an old fuel-limited evaluator succeeds. Introduce semantic leaf contracts, with the existing full-site checker and the new composition as two producers, then make the high-level simulation consume those contracts.

The composition must retain the entire prefix/control witness, arbitrary gap and registers, correct label mapping, exact return frame, and positive residual progress. A wrong intermediate visit to a dispatcher is not sufficient. The final theorem should cover every source leaf and guarded outcome, then reuse the existing runtime extension and endpoint. The measured opportunity is to stop recomputing a decoder proof already available in the kernel, not to replace proof obligations by Python execution.

5 Acceptance order and reproduction

First reconcile the current Rocq facade with its exact checked closure. Then port Lean's semantic bisimulation adapter to Rocq, keeping lookup-layout audits separate from semantic soundness. In parallel, compile Lean's direct fresh-label seed bridge using the existing Rocq argument. Finally compose the certified decoder with residual contracts and measure the full pipeline. None requires changing the Turing machine or weakening the public endpoint.

The accompanying packet preserves pinned source inputs, inspected-record summaries, mathematical proposals and owned portable checks. Run the supplied verifier with Python 3, assertions enabled, and a fresh external work directory:

```
python -I -B verify.py --work /tmp/round11-check
```

The verifier checks payload integrity and reproduces the stated source-text and symbolic audits. It does not run received build scripts, compile Lean or Rocq, or promote hashes and finite checks into kernel theorems. Exact compiler records remain historical evidence at their own source pins. New theorem compilation, dependency closure checks, assumption reports and controlled build measurements are the remaining acceptance gates.